



Arquitectura y Componentes Esenciales de los LLM

Fundamentos de Arquitectura LLM · Sesión 1 de 10

Atención, tokens, embeddings y ventana de contexto para aplicaciones
empresariales

www.bsginstitute.com

Andrés Felipe Rojas Parra

- Ingeniero electrónico con mas de 25 años de experiencia
- PGP Artificial Intelligence & Machine Learning at Texas University – Austin, TX
- Master en Inteligencia Artificial y Big Data – IEBSchool
- Estudiante de PGP en Cloud Computing at Texas University - Austin, TX
- Texas Executive Education Program – McCombs School of Business – Decision Science and AI Immersion Program
- Especialista en MLOps de la Universidad de Duke
- Estudiante de Deep Learning for Healthcare Specialization
- Director de Inteligencia Artificial e Innovacion Universidad EAN/Colombia



anrojas@universidadean.edu.co / arojaspa@outlook.com / arojaspa76



@arojaspa



<https://www.linkedin.com/in/arojaspa/>



+573005906373



CAPÍTULO 1

SESIÓN 1

3 HORAS

Arquitectura y Componentes Esenciales de los LLM

Transformers · Tokens · Embeddings · Contexto · Capacidades y Limitaciones



Token



Embed



Attn



Ctx



LLM

Agenda — Sesión 1

Tres temas clave • 3 horas de fundamentos con práctica

01

60 min

Arquitectura Transformer

- ▶ ¿Qué es un LLM?
- ▶ Mecanismo de Self-Attention (Q·K·V)
- ▶ Tokens y tokenización BPE
- ▶ Tipos de arquitectura (Encoder/Decoder)

02

60 min

Embeddings y Contexto

- ▶ ¿Qué son los embeddings?
- ▶ Similitud coseno y búsqueda semántica
- ▶ Ventana de contexto y sus límites
- ▶ Estrategias para contextos largos

03

60 min

Capacidades y Limitaciones

- ▶ Capacidades: generación, código, razonamiento
- ▶ Limitaciones: alucinaciones, knowledge cutoff
- ▶ Comparativa de modelos en producción
- ▶ Demo en vivo: Ollama + FastAPI + React



Break 10 min entre Tema 1 y Tema 2

¿Qué es un LLM?

Tema 1 · Large Language Model — Modelo de Lenguaje Extenso

■ Definición Técnica

- ✗ Modelo de IA preentrenado con trillones de tokens de texto
- ✗ Red neuronal con millones o billones de parámetros ajustables
- ✗ Aprende patrones del lenguaje — NO memoriza respuestas
- ✗ Genera texto token a token usando probabilidades
- ✗ Basa su "conocimiento" en los datos de entrenamiento

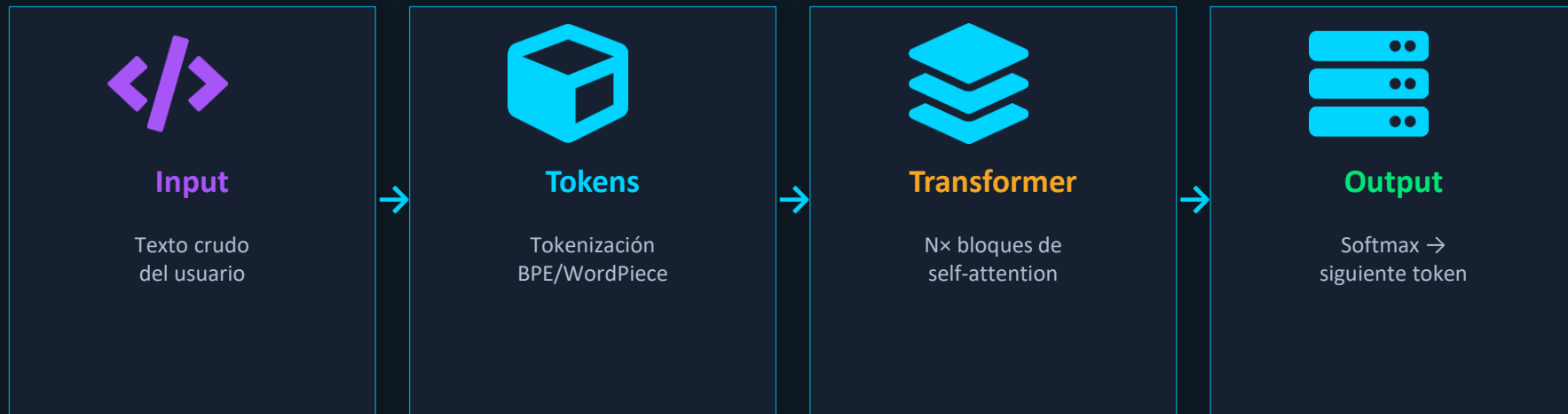


◆ Cómo Funciona

- ✓ Input → Tokenizar → Embeddings → Transformer → Softmax → Token
- ✓ Cada predicción usa TODOS los tokens previos en el contexto
- ✓ Temperatura controla la aleatoriedad de la selección
- ✓ Few-shot: aprende nuevas tareas con 2-3 ejemplos en el prompt
- ✓ Sin reentrenamiento ni fine-tuning para tareas nuevas

Arquitectura Transformer: "Attention Is All You Need"

Tema 1 · Vaswani et al. 2017 — la arquitectura base de todos los LLMs modernos



Componentes Clave del Transformer

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY ./app ./app
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0"]
```

Tipos de Arquitectura

Encoder-Only (BERT)

Decoder-Only (GPT, Llama, Mistral)

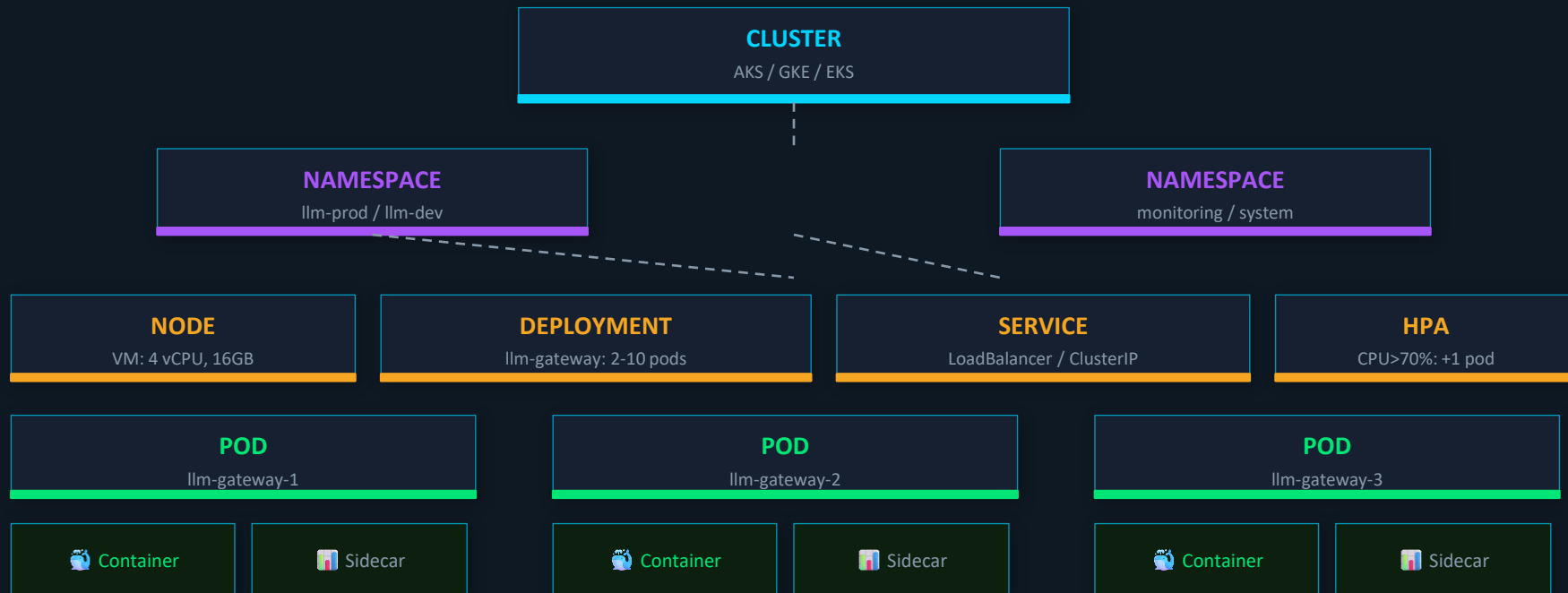
Encoder-Decoder (T5, BART)

Decoder-Only con MoE (Mixtral)

- Clasificación, NER, embeddings
- Generación de texto ← NUESTRO FOCO
- Traducción, resumen
- Escala masiva con costo controlado

Tokens: La Unidad Mínima del LLM

Tema 1 · El LLM no ve palabras — ve tokens (unidades sub-léxicas)



Self-Attention: el Corazón del Transformer

CONCEPTO CLAVE

Tema 1 · El mecanismo Q·K·V que permite a cada token "ver" a todos los demás



Mecanismo Q · K · V (Query, Key, Value)

```
# Para la frase: "El gato persigue al ratón"

# "gato" puede atender a "persigue" con peso 0.8

# y a "ratón" con peso 0.6

# Esto captura dependencias de largo alcance

Attention(Q,K,V) = softmax(QK^T / sqrt(d_k)) · V

# Q = lo que busco | K = lo que tengo | V = lo que entrego

# Multi-Head: N cabezas en paralelo (8-96 en GPT-4)

# cada cabeza aprende distintos tipos de relaciones
```

Tokens y Tokenización BPE

"transformers are amazing" → 6 tokens: [trans][form][ers][are][amaz][ing]
Español usa ~30% más tokens que inglés
100k tokens BPE: Byte-Pair Encoding — fusión iterativa
Impacto: más tokens = mayor costo de API \$0.15/M tokens → 500 palabras
ES ≈ 700 tokens ≈ \$0.0001



Ollama: Prueba Local Inmediata

CMD

```
ollama pull llama3.2 # descargar modelo 2GB
```

CMD

```
ollama run llama3.2 # chat interactivo
```

API

```
localhost:11434/api/  
generate # REST API directa
```

API

```
python  
examples/01_basic_ap  
i_call.py # primer script del curso
```

API

```
python  
examples/02_tokeniza  
tion.py # explorar tokens
```

```
@app.post("/chat")  
async def chat(req: ChatRequest):  
    resp = await ollama_client.chat(  
        model=req.model,  
        messages=req.messages)  
    return ChatResponse(**resp)
```


Ventana de Contexto: Límites del LLM

Tema 2 · Portabilidad como principio: build once, deploy anywhere



💡 Mismo Dockerfile para las 3 nubes — solo cambian las credenciales y el registry de destino

CAPÍTULO 1

TEMA 2

3 HORAS

Embeddings y Contexto

Representación semántica del texto como vectores numéricos



Vec



Cos



RAG



Sim



NLP

¿Qué son los Embeddings?

Tema 2 · Representaciones vectoriales que capturan significado semántico

■ Definición

✗ Un embedding transforma texto en un vector de N números flotantes

✗ nomic-embed-text → vector de 768 dimensiones

✗ text-embedding-3-large (OpenAI) → 3,072 dimensiones

✗ Textos similares → vectores cercanos en el espacio

✗ Basa su "conocimiento" en los datos de entrenamiento



◆ Similitud Coseno

✓ "automóvil" ↔ "carro" → similitud 0.94 (sinónimos)

✓ "IA" ↔ "artificial intelligence" → similitud 0.89 (multilingual)

✓ "pizza" ↔ "física cuántica" → similitud 0.31 (sin relación)

✓ $\cos(\theta) = (A \cdot B) / (|A| \times |B|) \in [-1, 1]$

✓ Búsqueda semántica: encontrar docs por significado, no por palabras

Embeddings en Producción: RAG y Búsqueda Semántica

Tema 1 · El mecanismo Q·K·V que permite a cada token "ver" a todos los demás



Pipeline RAG en 4 Pasos

```
# Para la frase: "El gato persigue al ratón"

# "gato" puede atender a "persigue" con peso 0.8

# y a "ratón" con peso 0.6
```

```
# 2. Query: buscar documentos relevantes

query_emb = get_embedding(user_question)

top_docs = vector_db.search(query_emb, k=3)
```

Aritmética de Embeddings

"transformers are amazing" → doctor - hombre + mujer ≈ doctora
 París - Francia + Colombia ≈ Bogotá
 El espacio vectorial captura relaciones semánticas
 Ejercicio: python examples/03_embeddings.py → nomic-embed-text vía Ollama (gratis)
 Ejecutar: ollama pull nomic-embed-text Ver similitud en tiempo real en la app ReactTab "Embeddings" → pares predefinidos + custom



Usos Empresariales de Embeddings

Caso

Chatbot sobre PDFs internos (RAG)

Búsqueda semántica en catálogo de productos

Caso

Clasificación automática de tickets de soporte

Detección de duplicados en base de conocimiento

Cmd

```
ollama pull nomic-embed-text
```

modelo de embeddings local

Cmd

```
python examples/03_embeddings.py
```

demo similitud coseno

Cmd

```
python examples/04_context_window.py
```

ventana de contexto

```
@app.post("/chat")
async def chat(req: ChatRequest):
    resp = await ollama_client.chat(
        model=req.model,
        messages=req.messages)
    return ChatResponse(**resp)
```

CAPÍTULO 1

TEMA 3

3 HORAS

Capacidades y Limitaciones

¿Qué pueden (y NO pueden) hacer los LLMs en producción?



Token



Embed



Attn



Cty



LLM

Capacidades de los LLMs Modernos

Tema 3 · Lo que los LLMs hacen bien — casos de uso validados en producción



01



Generación de Texto

Redactar, resumir, traducir, adaptar tono. Escala masiva con calidad consistente.



02

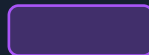


Conversación Natural

Diálogos coherentes multi-turno. Memoria de contexto hasta 128K tokens.



03



Generación de Código

Python, JS, SQL, Shell. Debuggear, refactorizar, documentar. GitHub Copilot x10.



04



Razonamiento Step-by-Step

Chain-of-Thought: resolver problemas complejos dividiéndolos en pasos.



05



Extracción de Información

NER, clasificación, structured output (JSON). Procesar documentos no estructurados.



06



Multilenguaje

Llama 3.2 maneja 8+ idiomas. Traducción, detección, respuesta en el idioma del usuario.



07



Few-Shot Learning

2-3 ejemplos en el prompt y el modelo aprende la tarea. Sin fine-tuning.

Limitaciones Críticas en Producción

Tema 3 · Lo que los LLMs NO pueden hacer — errores comunes que cuestan caro



01



Alucinaciones

Genera información plausible pero falsa. Citas, datos, URLs inventadas. Validar siempre.



02



Knowledge Cutoff

Conocimiento congelado en fecha de entrenamiento. Sin acceso a info reciente.



03



Aritmética Compleja

Suma básica OK, álgebra lineal o cálculo = propenso a errores. Delegar a herramientas.



04



Razonamiento Lógico Estricto

Falla en silogismos formales y problemas de lógica espacial precisa.



05



Inconsistencia

Temperature > 0 → respuestas distintas a la misma pregunta. Validar en prod.



06



Memoria Persistente

Olvida todo al terminar la conversación. Requiere BD externa o RAG para "recordar".



07



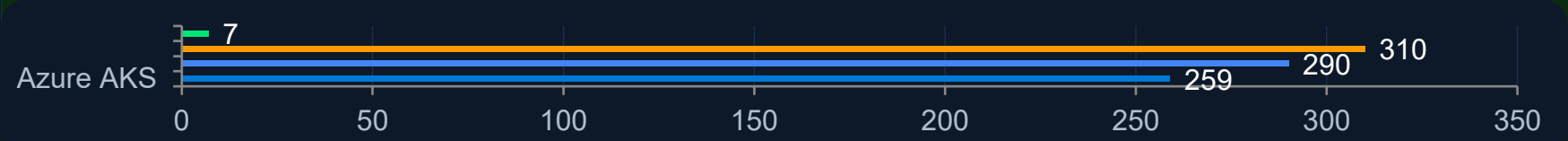
Privacidad de Datos

Datos enviados a APIs cloud. Usar Ollama local o VPC endpoints para datos sensibles.

Comparativa de Modelos LLM 2025

Tema 3 · Escenario real: LLM Gateway para empresa LATAM mediana

Componente	Azure (AKS)	GCP (GKE)	AWS (EKS)	Ollama Local
Cómputo (nodos)	\$120	\$98	\$105	\$0
Control Plane K8s	\$0	\$73	\$73	\$0
API LLM (tokens)	\$85	\$72	\$80	\$0
Storage + Registry	\$18	\$15	\$17	\$5
Red (egress)	\$20	\$18	\$20	\$2
Monitoreo	\$16	\$14	\$15	\$0
TOTAL/MES	\$259	\$290	\$310	~\$7



Demo en Vivo: Ollama + FastAPI + React

DEMO EN VIVO

Tema 1 · El mecanismo Q·K·V que permite a cada token "ver" a todos los demás



Paso 1: Instalar y correr Ollama

```
# Para la frase: "El gato persigue al ratón"

# "gato" puede atender a "persigue" con peso 0.8

# y a "ratón" con peso 0.6
```

```
ollama serve          # servidor en :11434
```

```
# Windows: descargar OllamaSetup.exe
```

```
# desde ollama.com/download/windows
```

```
ollama pull nomic-embed-text # embeddings
```

```
ollama list # verificar modelos instalados
```

Paso 2: Levantar el Backend FastAPI

```
"transformers are amazing" → pip install -r requirements.txtuvicorn
main:app --reload
# API en http://localhost:8000# Docs: http://localhost:8000/docsPaso 3:
Levantar el Frontend React
cd session1/frontend && npm installnpm run dev #
http://localhost:5173→ Tab Chat: probar system prompts y temperatura
```



Endpoints Disponibles

POST

/api/chat

Chat con LLM local

POST

/api/tokenize

Contar y ver tokens

POST

/api/similarity

Similitud coseno

POST

/api/embed

Generar embeddings

POST

/api/models

Modelos disponibles

```
@app.post("/chat")
async def chat(req: ChatRequest):
    resp = await ollama_client.chat(
        model=req.model,
        messages=req.messages)
    return ChatResponse(**resp)
```



Resumen de la Sesión 1



Transformer: Self-Attention Q·K·V — cada token ve a todos los demás



Tokens: unidad mínima del LLM — español usa ~30% más que inglés



Embeddings: vectores semánticos — similitud coseno mide significado



Contexto: límite de "memoria" del modelo — 4K a 1M tokens según modelo



Capacidades: generación, código, razonamiento, extracción, multilinguaje



Limitaciones: alucinaciones, knowledge cutoff, memoria, aritmética

★ Recursos del Curso

Repositorio GitHub: github.com/bsg-institute/session1-llm-fundamentals

Guía Ollama: [docs/OLLAMA_SETUP.md](#) en el repositorio

Ejercicios prácticos: [docs/EXERCISES.md](#) — 4 ejercicios guiados

Próxima sesión: [Sesión 2 — Prompt Engineering y RAG Avanzado](#)